Lecture 22: B-Splines: Definition, Properties, Interpolation

Math 6610

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Today's Goals

- B-splines: Cox–de Boor recursion, first examples
- Five properties of B-splines: the wish list, proved
- Evaluate all B-splines at x in $O(k^2)$ operations
- Clamped knots: the N_{ik} form a basis; de Boor points
- Spline interpolation: a banded, well conditioned system
- The payoff, measured: B-splines vs truncated powers

Recap from Lecture 21

Splines. $a = x_0 < \cdots < x_n = b$; $\mathcal{S}_{k,\Delta}$: $S \in C^{k-2}[a,b]$, $S \in \prod_{k-1}$ on each $[x_{j-1}, x_j]$; $\dim \mathcal{S}_{k,\Delta} = k + n - 1$.

Truncated power basis: $1, x, \ldots, x^{k-1}, (x - x_1)_+^{k-1}, \ldots, (x - x_{n-1})_+^{k-1}$. A basis, but

- ill-conditioned: $\kappa_\infty(T) \approx 9.2 \cdot 10^7$ at $n = 80$ (not-a-knot cubic);
- global support: each function is nonzero up to b.

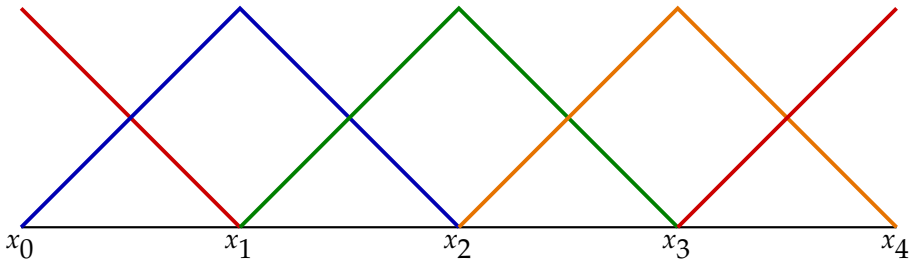
Wish list for a better basis:

- local: each function nonzero on at most k consecutive intervals;
- nonnegative, and summing to 1 on $[a,b]$;
- well conditioned, cheap and stable to evaluate.

Today: B-splines, which meet all three.

The Simplest Case: Hat Functions ($k = 2$)

On $x_0, \ldots, x_4$: N_{i2} is 1 at x_{i-1}, 0 at the other knots, linear in between.



- Local: each hat lives on $\leqslant 2$ intervals.
- Nonnegative, and the hats sum to 1.
- Interpolation needs no linear system:

$$S(x) = \sum_{i=1}^{n+1} y_{i-1} N_{i2}(x) \quad \Longrightarrow \quad S(x_{i-1}) = y_{i-1}, \ i = 1, \ldots, n+1.$$

Goal: the same for any order k.

Knots for B-Splines

A *knot sequence* may repeat values:

$$\tau_1 \leqslant \tau_2 \leqslant \cdots \leqslant \tau_l, \qquad \tau_i \in \mathbb{R}. \tag{1}$$

Definition (*Multiplicity*). The *multiplicity* of a knot value is the number of times it appears in (1).

Definition (*Clamped knots for $\mathcal{S}_{k,\Delta}$*). End points k times, interior knots once:

$$\overbrace{\underbrace{x_0}_{\tau_1=\cdots=\tau_k}}^{k \text{ times}} < \underbrace{x_1}_{\tau_{k+1}} < \cdots < \underbrace{x_{n-1}}_{\tau_{k+n-1}} < \overbrace{\underbrace{x_n}_{\tau_{k+n}=\cdots=\tau_{2k+n-1}}}^{k \text{ times}}$$

Count: $l = 2k + n - 1$, and $l - k = k + n - 1 = \dim \mathcal{S}_{k,\Delta}$.

Example. $k = 2$, $n = 4$: $\tau = (x_0, x_0, x_1, x_2, x_3, x_4, x_4)$, $l = 7$: 5 hats (the two doubled end points give the two half hats).

B-Splines: Definition

Definition (*B-splines; Cox–de Boor recursion*). For the knots (1), the B-splines N_{ij} (index i, order j):

$$N_{i1}(x) := \begin{cases} 1, & \tau_i \leqslant x < \tau_{i+1}, \\ 0, & \text{otherwise}, \end{cases} \qquad i = 1, \ldots, l-1,$$

and for $j = 2, \ldots, k,\ \ i = 1, \ldots, l-j$:

$$N_{ij}(x) := \frac{x - \tau_i}{\tau_{i+j-1} - \tau_i} N_{i,j-1}(x) + \frac{\tau_{i+j} - x}{\tau_{i+j} - \tau_{i+1}} N_{i+1,j-1}(x). \tag{2}$$

In words: order j = (rising ramp) $\times$ left neighbour $+$ (falling ramp) $\times$ right neighbour, both of order $j-1$.

B-Splines: Two Conventions

1. A term with denominator 0 is 0. (Nothing is lost: $\tau_i = \tau_{i+j-1}$ forces $N_{i,j-1} \equiv 0$.)
2. $N_{ik}(\tau_l) := \lim_{x \to \tau_l^-} N_{ik}(x)$, so the right end point $b = \tau_l$ is covered.

Example (*Convention 1 at work: a half hat*). $k = 2$, clamped: $\tau_1 = \tau_2 = x_0 < \tau_3 = x_1$. In (2) with $i = 1$, $j = 2$:

$$N_{12}(x) = \underbrace{\frac{x - \tau_1}{\tau_2 - \tau_1} N_{11}(x)}_{\text{denominator } 0\text{: term } 0} + \frac{\tau_3 - x}{\tau_3 - \tau_2} N_{21}(x) = \begin{cases} \dfrac{x_1 - x}{x_1 - x_0}, & x_0 \leqslant x < x_1, \\ 0, & \text{otherwise.} \end{cases}$$

This is the left half hat: 1 at x_0, 0 at x_1. (And $N_{11} \equiv 0$ anyway: its box $[\tau_1, \tau_2)$ is empty.)

Orders 1 and 2 by Hand

Equally spaced knots $\tau_i = i$ (no repeats).
Order 1: boxes, $N_{i1} = 1$ on $[i, i+1)$, 0 otherwise.
Order 2: (2) with $j = 2$; both denominators are 1:

$$N_{i2}(x) = (x-i)\,N_{i1}(x) + (i+2-x)\,N_{i+1,1}(x) = \begin{cases} x-i, & i \leqslant x < i+1, \\ i+2-x, & i+1 \leqslant x < i+2, \\ 0, & \text{otherwise.} \end{cases}$$

Up-ramp on the left box $+$ down-ramp on the right box $=$ a hat.

Remark. The weight $\frac{x-\tau_i}{\tau_{i+j-1}-\tau_i}$ rises from 0 to 1 on $[\tau_i, \tau_{i+j-1}]$; the other falls from 1 to 0 on $[\tau_{i+1}, \tau_{i+j}]$. So each order adds one interval to the support (the set where the function is nonzero).

Order 3 by Hand

Same knots, $j = 3$; both denominators are 2. With $s = x - i$:

$$N_{i3}(x) = \tfrac{s}{2} N_{i2}(x) + \tfrac{3-s}{2} N_{i+1,2}(x) = \begin{cases} \frac{1}{2}s^2, & 0 \leqslant s < 1, \\ \frac{1}{2}(-2s^2 + 6s - 3), & 1 \leqslant s < 2, \\ \frac{1}{2}(3-s)^2, & 2 \leqslant s < 3, \\ 0, & \text{otherwise.} \end{cases}$$

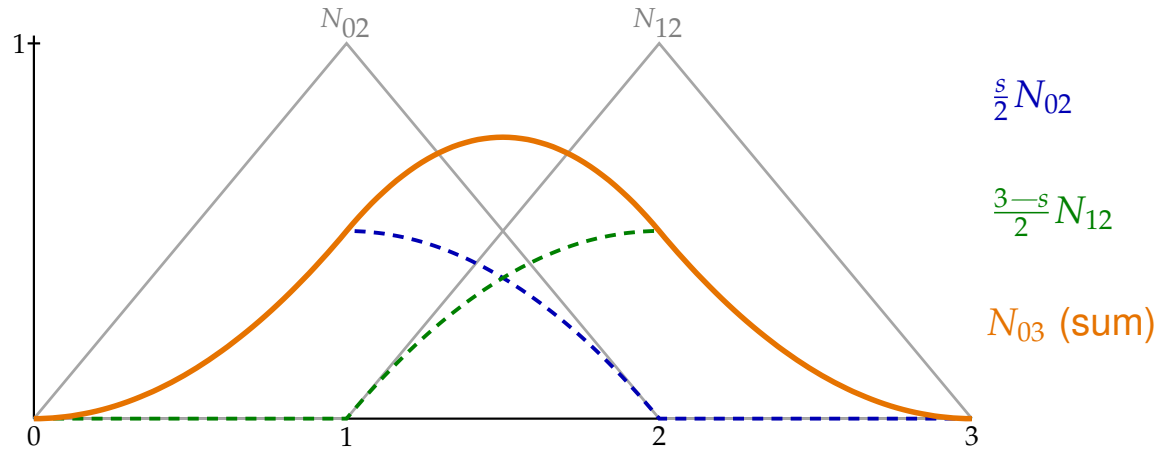
(Middle piece: $\frac{s}{2}(2-s) + \frac{3-s}{2}(s-1)$.)

Check the wish list:

- Smooth: C^1 at $s = 1$ (values $\frac{1}{2} = \frac{1}{2}$, slopes $1 = 1$); at $s = 2$ by symmetry $s \mapsto 3 - s$.
- Local: support $[i, i+3]$, 3 intervals.
- Sum 1: at $x = 1.5$ only $N_{-1,3}, N_{0,3}, N_{1,3}$ are nonzero ($s = 2.5, 1.5, 0.5$): $\frac{1}{8} + \frac{3}{4} + \frac{1}{8} = 1$.

Order 3: The Recursion as a Picture

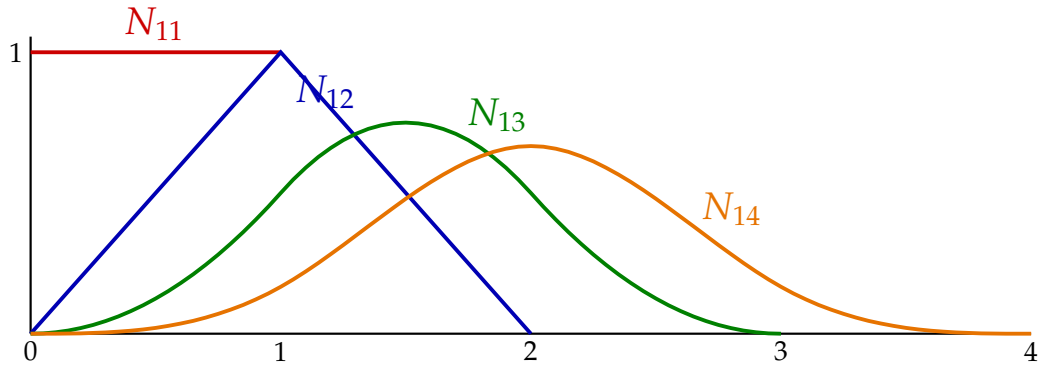
$i = 0$, $\tau_i = i$: $N_{03} = \frac{s}{2} N_{02} + \frac{3-s}{2} N_{12}$, $s = x$.



- Gray: the two hats of order 2. Dashed: each hat times its ramp.
- The sum (orange) is smoother than either term: the corners of the hats cancel, leaving C^1.
- Peak value $N_{03}(1.5) = \frac{3}{4}$. (Check: $\frac{1.5}{2} \cdot 0.5 + \frac{1.5}{2} \cdot 0.5 = 0.75$.)

Orders 1 to 4 on Equally Spaced Knots

$N_{11}, N_{12}, N_{13}, N_{14}$ for $\tau_i = i$: supports $[0,1]$, $[0,2]$, $[0,3]$, $[0,4]$.



- Each order: one more interval of support, one more continuous derivative.
- N_{14}, the cubic B-spline: C^2, with $N_{14} = N'_{14} = N''_{14} = 0$ at both ends of its support.

Five Properties

Proposition. Let (1) hold and $1 \leqslant i \leqslant l-k$.

(a) *Support:* $N_{ik}(x) = 0$ for $x \notin [\tau_i, \tau_{i+k})$.

(b) *Nonnegative:* $N_{ik}(x) \geqslant 0$ for all $x \in \mathbb{R}$.

(c) *Partition of unity:* if $\tau_k < \tau_{l-k+1}$, then $\sum_{i=1}^{l-k} N_{ik}(x) = 1$ for $x \in [\tau_k, \tau_{l-k+1})$.

(d) *Smoothness:* at a knot value of multiplicity $m \leqslant k$, N_{ik} is C^{k-1-m} (for $m = k$ it may jump).

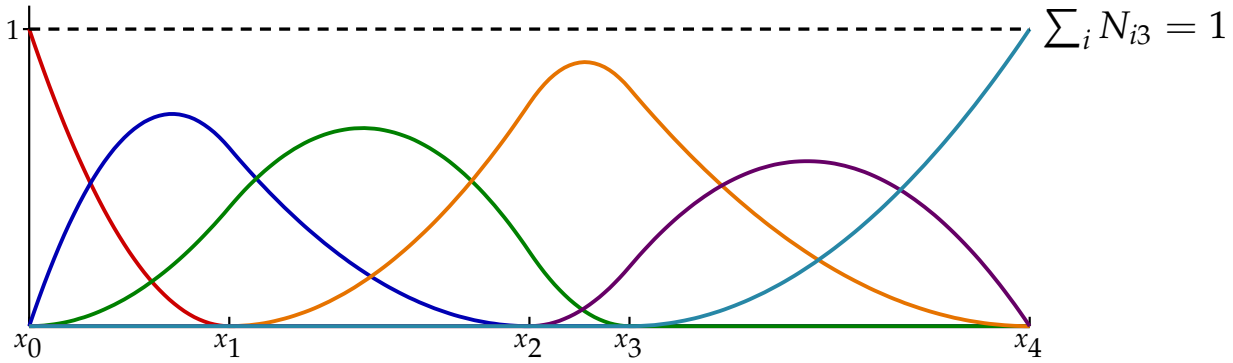
(e) *Piecewise polynomial:* $N_{ik} \in \prod_{k-1}$ on each $[\tau_j, \tau_{j+1})$.

Remark (*Clamped knots*). $[\tau_k, \tau_{l-k+1}] = [a, b]$, so (c) holds on all of $[a, b]$ (at b by convention 2). Interior knots are simple ($m = 1$), so by (d), (e): every $N_{ik} \in \mathcal{S}_{k,\Delta}$.

Remark (*Proofs*). Induction on the order, using the recursion (2); omitted here (C. de Boor, *A Practical Guide to Splines*, Ch. IX). We check them on pictures instead.

Five Properties: A Picture

$k = 3$, clamped knots $0, 0, 0, 1, 2.5, 3, 5, 5, 5$ (uneven on purpose):



- (a) Each N_{i3} lives on at most 3 intervals.
- (b) All curves stay $\geqslant 0$. (c) Dashed: their sum is 1 everywhere on $[0, 5]$.
- (d), (e) Quadratic pieces, joined with matching slopes (C^1) at the simple knots $1, 2.5, 3$.

What the Properties Buy

For $S(x) = \sum_i \gamma_i N_{ik}(x)$ on clamped knots:

property	consequence
(a) local support	$\leqslant k$ terms at each x: cheap evaluation, banded matrices, local edits
(b) $\geqslant 0$, (c) sum 1	$S(x)$ = weighted average of the γ_i
(d), (e)	$N_{ik} \in \mathcal{S}_{k,\Delta}$; in fact a basis

Plan for today:

1. evaluate (uses (a));
2. basis and de Boor points (uses (b)–(e));
3. interpolate (uses all).

B-splines (and NURBS) are the standard curves in CAD and graphics (Lecture 17: fonts, the teapot).

B-Splines in Matlab: The Recursion

(2) translates line by line (`tau(i)` $= \tau_i$):

```
function v = bspl(tau, i, j, x)
% N_{ij}(x) by the recursion; x may be a vector
if j == 1
  v = double(tau(i) <= x & x < tau(i+1));  return
end
v  = zeros(size(x));
d1 = tau(i+j-1) - tau(i);  d2 = tau(i+j) - tau(i+1);
if d1 > 0                                % d1 = 0: term is 0
  v = v + (x - tau(i))/d1 .* bspl(tau, i, j-1, x);
end
if d2 > 0
  v = v + (tau(i+j) - x)/d2 .* bspl(tau, i+1, j-1, x);
end
```

B-Splines in Matlab: The Recursion

Clamped cubic B-splines on $0, 1, \ldots, 5$ (picture later):

```
tau = [0 0 0 0 1 2 3 4 5 5 5 5];  k = 4;    % 8 B-splines
xx = linspace(0, 5 - 1e-12, 501);            % x < b = 5
for i = 1:numel(tau)-k
  plot(xx, bspl(tau, i, k, xx)); hold on
end
```

Check property (c):

```
s = 0;  for i = 1:8, s = s + bspl(tau, i, k, xx); end
max(abs(s - 1))                               % about 1e-16
```

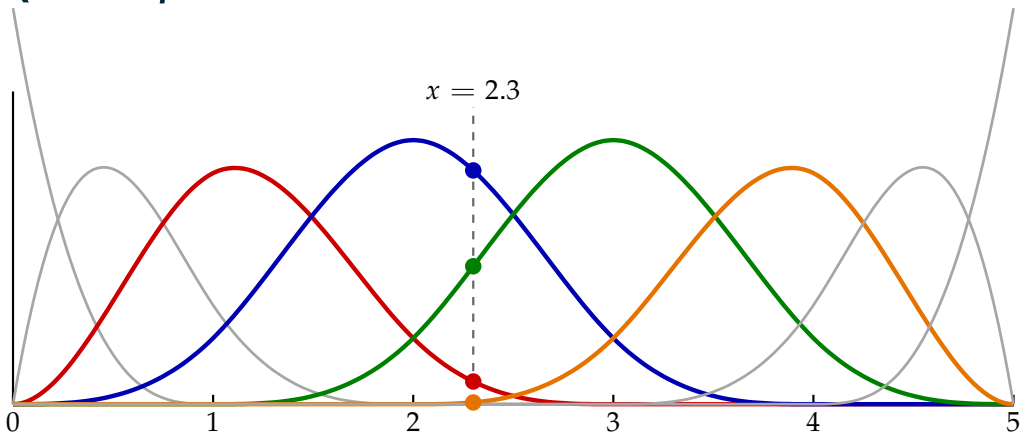
Remark. Correct, but wasteful: one N_{ik} costs up to $2^k - 1$ calls, and the same lower-order values are recomputed. Next: compute each value once.

Which B-Splines Are Nonzero at x

Proposition. Let $\tau_m \leqslant x < \tau_{m+1}$. For each order $j = 1, \ldots, k$, only $N_{m-j+1,j}, \ldots, N_{m,j}$ (j of them) can be nonzero at x.

Reason: by (a), $N_{ij}(x) \neq 0$ needs $\tau_i \leqslant x < \tau_{i+j}$.

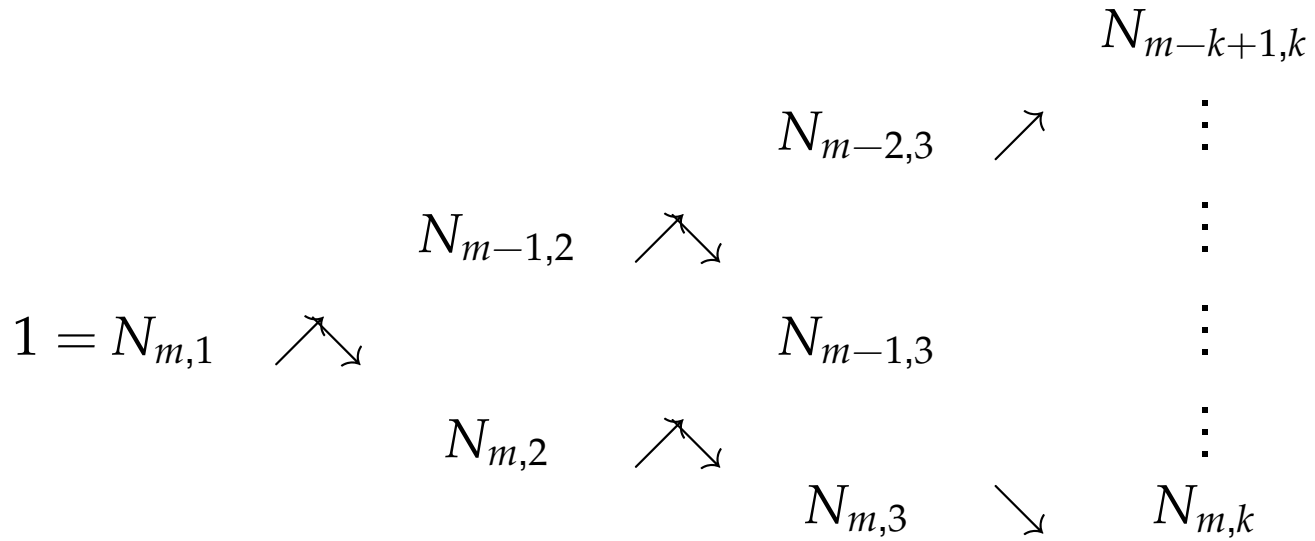
Example (*Clamped cubics on* $0, 1, \ldots, 5$, $x = 2.3$, $m = 6$).



Only $N_{34}, \ldots, N_{64}$ are nonzero; the other 4 (gray) vanish. (Check: the 4 values sum to 1.)

Shared Lower-Order Values

By (2), each value of order j feeds two values of order $j+1$. At x, only a triangle of the table is needed:



- `bspl` recomputes shared entries (e.g. $N_{m-1,2}$ feeds both $N_{m-2,3}$ and $N_{m-1,3}$).
- Fix: fill the triangle column by column, each entry once.

The Triangular Scheme: Algorithm

Remark (*Algorithm*). Input: knots (1), order k, point x, index m with $\tau_m \leqslant x < \tau_{m+1}$, $k \leqslant m \leqslant l-k$.

$N := (1)$ (order 1: $N_{m,1}(x)$)

for $j = 1, \ldots, k-1$ (order $j \to j+1$; $N_r = N_{m-j+r,\,j}(x)$)

  $saved := 0$

  for $r = 1, \ldots, j$

    $t_R := \tau_{m+r}, \;\; t_L := \tau_{m+r-j}, \;\; term := N_r/(t_R - t_L)$

    $N_r^{\text{new}} := saved + (t_R - x)\, term$

    $saved := (x - t_L)\, term$

  $N_{j+1}^{\text{new}} := saved, \;\; N := N^{\text{new}}$

Output: $N = \big(N_{m-k+1,k}(x), \ldots, N_{m,k}(x)\big)$.

The Triangular Scheme

1. It is (2). With $i = m - j + r$: $t_L = \tau_i$, $t_R = \tau_{i+j}$. The value N_{ij} feeds two entries of order $j+1$, with one shared denominator:

$$N_{ij} \longrightarrow \begin{cases} N_{i-1,j+1} & \text{weight } \dfrac{t_R - x}{t_R - t_L} \quad (\text{added to } N_r^{\text{new}}), \\ N_{i,j+1} & \text{weight } \dfrac{x - t_L}{t_R - t_L} \quad (\text{carried in } \mathit{saved}). \end{cases}$$

2. No division by 0*:* $t_L \leqslant \tau_m < \tau_{m+1} \leqslant t_R$, so $t_R - t_L > 0$.
3. No cancellation: $x - t_L \geqslant 0$ and $t_R - x > 0$, so every term is $\geqslant 0$; sums of nonnegative numbers are computed accurately.

The Triangular Scheme in Matlab

```
function N = bsplvb(tau, k, x, m)
% N_{m-k+1,k}(x), ..., N_{m,k}(x) for tau(m) <= x < tau(m+1)
N = 1;                                  % order 1: N_{m,1}(x) = 1
for j = 1:k-1                           % order j -> order j+1
  Nnew = zeros(j+1, 1);  saved = 0;
  for r = 1:j                           % N(r) = N_{m-j+r,j}(x)
    tR = tau(m+r);  tL = tau(m+r-j);
    term = N(r)/(tR - tL);
    Nnew(r) = saved + (tR - x)*term;
    saved = (x - tL)*term;
  end
  Nnew(j+1) = saved;  N = Nnew;
end
```

- *Locate m:* `m = find(tau <= x, 1, 'last');` at $x = b$ use `m = l-k` (convention 2).
- *Cost:* 7 flops per inner step, $1 + \cdots + (k-1)$ steps: $\frac{7}{2}k(k-1)$ flops. Cubic: 42, independent of n.

Example: The Triangle by Hand

$k = 3,\ \tau = (0, 0, 0, 1, 2, 2, 2),\ x = 0.5$: $\tau_3 = 0 \leqslant 0.5 < 1 = \tau_4$, so $m = 3$.

Order 1 $\to$ 2 ($j = 1$):

- $r = 1$: $t_R = \tau_4 = 1$, $t_L = \tau_3 = 0$, $term = 1$:

$$N_{2,2} = 0 + 0.5 \cdot 1 = 0.5, \qquad saved = N_{3,2} = 0.5 \cdot 1 = 0.5.$$

Order 2 $\to$ 3 ($j = 2$):

- $r = 1$: $t_R = \tau_4 = 1$, $t_L = \tau_2 = 0$, $term = 0.5$:

$$N_{1,3} = 0 + 0.5 \cdot 0.5 = 0.25, \qquad saved = 0.25.$$

- $r = 2$: $t_R = \tau_5 = 2$, $t_L = \tau_3 = 0$, $term = 0.25$:

$$N_{2,3} = 0.25 + 1.5 \cdot 0.25 = 0.625, \qquad N_{3,3} = 0.5 \cdot 0.25 = 0.125.$$

Example: The Triangle by Hand

The filled triangle:

$$
\begin{array}{ccc}
 & & N_{1,3} = 0.25 \\
 & N_{2,2} = 0.5 & \\
N_{3,1} = 1 & & N_{2,3} = 0.625 \\
 & N_{3,2} = 0.5 & \\
 & & N_{3,3} = 0.125
\end{array}
$$

- Partition of unity: $0.25 + 0.625 + 0.125 = 1$.
- Direct check: on $[0, 1]$ these knots give $N_{1,3}(x) = (1 - x)^2$, so $N_{1,3}(0.5) = 0.25$.
- 3 inner steps in total; each order-2 value was computed once.

Clamped Knots Give a Basis

Theorem (*proof omitted; de Boor, Ch. IX*). For clamped knots ($l = 2k + n - 1$), the B-splines N_{ik}, $i = 1, \ldots, k + n - 1$, form a basis of $\mathcal{S}_{k,\Delta}$.

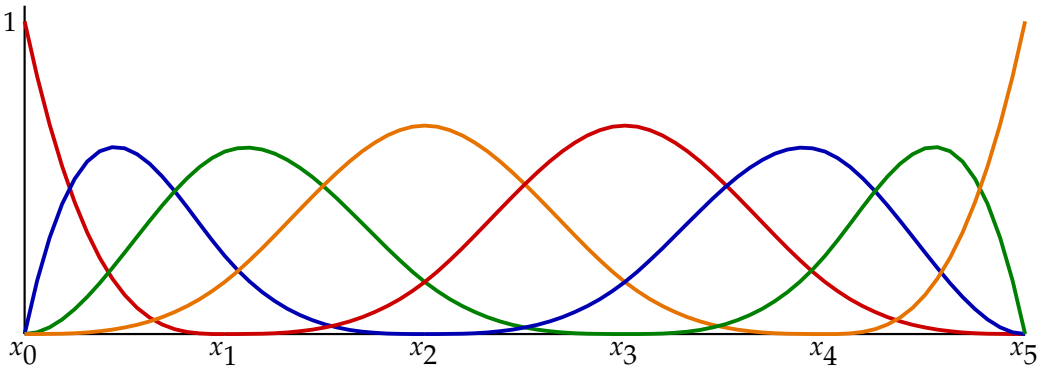
Remark (*Why it is plausible*). Each $N_{ik} \in \mathcal{S}_{k,\Delta}$ ("Five Properties"), and there are $k + n - 1 = \dim \mathcal{S}_{k,\Delta}$ of them (Lecture 21). What remains is linear independence.

So every $S \in \mathcal{S}_{k,\Delta}$ is, in exactly one way,

$$S(x) = \sum_{i=1}^{k+n-1} \gamma_i N_{ik}(x).$$

Clamped Cubic B-Splines

$k = 4$, $x_0, \ldots, x_5 = 0, 1, \ldots, 5$: knots $\tau = (0, 0, 0, 0, 1, 2, 3, 4, 5, 5, 5, 5)$, $l = 12$, 8 B-splines.



- End B-splines start at height 1: $N_{14}(x_0) = N_{84}(x_5) = 1$.
- $N_{i4} = 0$ outside $[\tau_i, \tau_{i+4})$ (a); at each x at most 4 are nonzero, and they sum to 1 (c).

De Boor Points

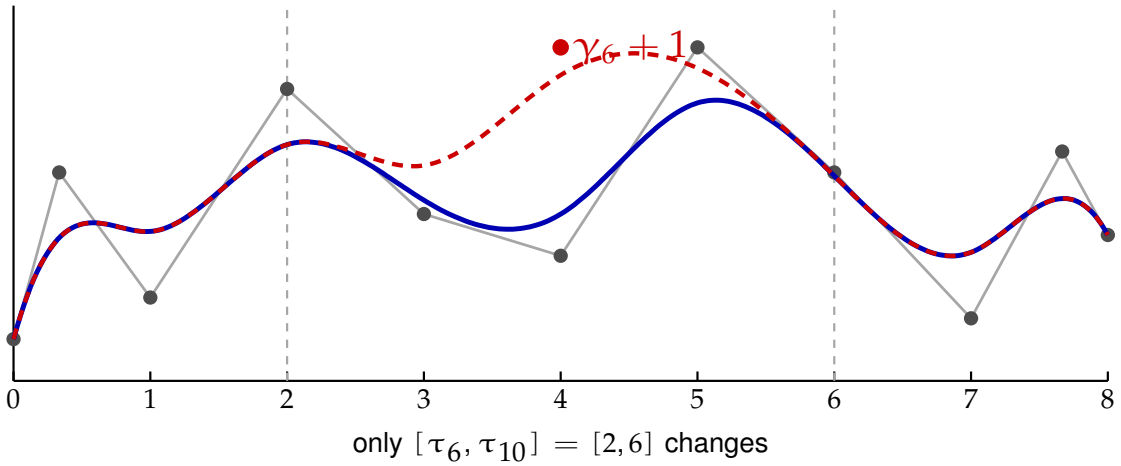
Definition (*de Boor points*). For $S = \sum_{i=1}^{k+n-1} \gamma_i N_{ik} \in \mathcal{S}_{k,\Delta}$, the coefficients $\gamma_1, \ldots, \gamma_{k+n-1}$ are the *de Boor points* of S.

Remark (*What (a)–(c) give*). At each $x \in [a, b]$, $S(x)$ is a weighted average of the γ_i (weights $N_{ik}(x) \geqslant 0$, sum 1). Hence:

- no overshoot: $\min_i \gamma_i \leqslant S(x) \leqslant \max_i \gamma_i$;
- stable: changing every γ_i by at most δ changes S by at most δ;
- local control: changing only γ_i changes S only on $[\tau_i, \tau_{i+k}]$ (by (a)).

De Boor Points: Local Control

Cubic, clamped knots on $0, 1, \ldots, 8$; 11 de Boor points (dots), each drawn at $\xi_i = (\tau_{i+1} + \tau_{i+2} + \tau_{i+3})/3$:



- Blue: S. It follows the polygon of de Boor points, smoothed.
- Red dashed: γ_6 raised by 1. S moves only on $[\tau_6, \tau_{10}] = [2, 6]$; outside, nothing changes.
- This is how CAD programs edit curves: drag one control point.

Interpolation: The Not-a-Knot Cubic

$k = 4$, $n \geqslant 3$. Drop x_1 and x_{n-1} as knots (Lecture 18):

$$\underbrace{x_0}_{\tau_1 = \cdots = \tau_4} < x_1 < \underbrace{x_2}_{\tau_5} < \cdots < \underbrace{x_{n-2}}_{\tau_{n+1}} < x_{n-1} < \underbrace{x_n}_{\tau_{n+2} = \cdots = \tau_{n+5}}$$

$l = n + 5$, so $n + 1$ B-splines: $S(x) = \sum_{i=1}^{n+1} \gamma_i N_{i4}(x)$.

Conditions $S(x_{r-1}) = y_{r-1}$, $r = 1, \ldots, n+1$:

$$M\gamma = y, \qquad M = [m_{ri}]_{r,i=1}^{n+1}, \quad m_{ri} = N_{i4}(x_{r-1}).$$

Row r = the B-splines evaluated at one point: at most 4 nonzeros, in consecutive columns (property (a)), computed by `bsplvb`.

Example: $n = 4$

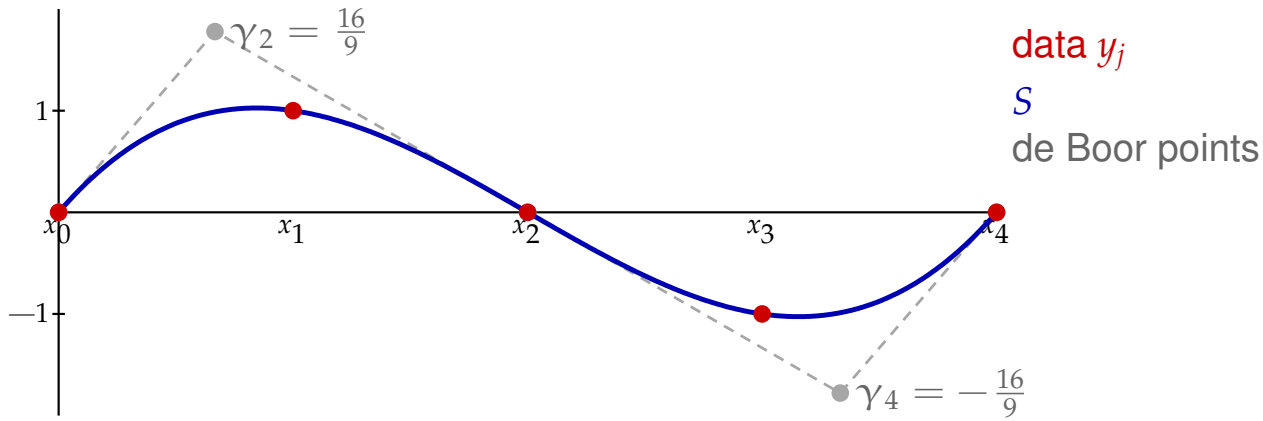
$x_j = j$, $j = 0, \ldots, 4$; $\tau = (0, 0, 0, 0, 2, 4, 4, 4, 4)$; 5 B-splines.

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ \frac{1}{8} & \frac{19}{32} & \frac{1}{4} & \frac{1}{32} & 0 \\ 0 & \frac{1}{4} & \frac{1}{2} & \frac{1}{4} & 0 \\ 0 & \frac{1}{32} & \frac{1}{4} & \frac{19}{32} & \frac{1}{8} \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

(each row sums to 1, by (c))

- Row 3 has only 3 nonzeros: at $x_2 = \tau_5$, N_{54} starts at 0.
- $y = (0, 1, 0, -1, 0)$ gives $\gamma = (0, \frac{16}{9}, 0, -\frac{16}{9}, 0)$. (Check, row 2: $(\frac{19}{32} - \frac{1}{32}) \cdot \frac{16}{9} = 1$.)
- $\frac{16}{9} > \max_j y_j = 1$: de Boor points are not the data (except at the ends).

Example: $n = 4$, A Picture



- S (blue) passes through all 5 data points (red).
- The de Boor points (gray, at ξ_i) overshoot the data: $\gamma_2 = \frac{16}{9} > 1$. They steer the curve; they are not on it, except at the two ends.

B-Spline Interpolation in Matlab

Build M row by row with `bsplvb`, then solve:

```
function [tau, g] = bsinterp(x, y)
% not-a-knot cubic spline through (x_j, y_j), in B-spline form
x = x(:);  y = y(:);  n = length(x) - 1;  k = 4;
tau = [x(1)*ones(1,4), x(3:n-1)', x(n+1)*ones(1,4)];  % l=n+5
l = numel(tau);  M = sparse(n+1, n+1);
for r = 1:n+1
  m = min(find(tau <= x(r), 1, 'last'), l-k);
  M(r, m-k+1:m) = bsplvb(tau, k, x(r), m);
end
g = M \ y;                                          % banded solve
```

- `x(1)` $= x_0$, so `x(3:n-1)` $= x_2, \ldots, x_{n-2}$.
- `min(..., l-k)`: at $x_n = b$, use the last nonempty interval (convention 2).

B-Spline Interpolation in Matlab

Evaluate $S(x) = \sum_i \gamma_i N_{i4}(x)$: 4 terms per point.

```
function s = bsval(tau, g, xx)
k = 4;  l = numel(tau);  s = zeros(size(xx));
for q = 1:numel(xx)
  m = min(find(tau <= xx(q), 1, 'last'), l-k);
  s(q) = g(m-k+1:m)' * bsplvb(tau, k, xx(q), m);
end
```

Compare with Matlab's `spline` (also not-a-knot):

```
x = 0:4;  y = [0 1 0 -1 0];
[tau, g] = bsinterp(x, y)              % g = [0 16/9 0 -16/9 0]'
xx = linspace(0, 4, 101);
max(abs(bsval(tau, g, xx) - spline(x, y, xx)))   % about 1e-15
```

Same interpolant (it is unique), two representations: piecewise coefficients vs de Boor points.

Solving with M

Cost. At most 4 nonzeros per row, consecutive: M is banded. Storage $O(n)$; banded LU $O(k^2 n)$ flops. (`M \ y` detects the band.)

Remark (*Facts, proofs omitted*).

1. *Schoenberg–Whitney:* M is nonsingular (invertible) if and only if all diagonal entries $N_{i4}(x_{i-1})$ are nonzero. Here they are.
2. M is *totally nonnegative* (every square submatrix has determinant $\geqslant 0$). For such matrices, Gaussian elimination without pivoting is backward stable (de Boor and Pinkus, 1977).

Conclusion: solvable, no pivoting, $O(n)$ work. Same lesson as the natural spline in Lecture 18.

Payoff: B-Splines vs Truncated Powers

Same interpolant (not-a-knot cubic, $n+1$ equally spaced points on $[0,1]$), two bases: T (truncated powers, Lecture 21), M (B-splines).

```
cond(T, inf),  cond(full(M), inf)
```

n	5	10	20	40	80
$\kappa_\infty(T)$	$2.9 \cdot 10^3$	$4.2 \cdot 10^4$	$4.9 \cdot 10^5$	$6.4 \cdot 10^6$	$9.2 \cdot 10^7$
$\kappa_\infty(M)$	4.6	4.8	4.8	4.8	4.8

- $\kappa_\infty(T)$: $\times 14$ per doubling of n (about n^4). $\kappa_\infty(M)$: below 5 for all n.
- Same space, same spline: the basis decides the computation.

Matlab's Own Tools

task	Matlab
not-a-knot cubic, piecewise form	`pp = spline(x, y)`
evaluate piecewise form	`ppval(pp, xx)`
knots with end multiplicity k	`tau = augknt(knots, k)`
matrix M	`M = spcol(tau, k, x)`
interpolant in B-spline form	`sp = spapi(tau, x, y)`
evaluate B-spline form	`fnval(sp, xx)`

(All but `spline`, `ppval`: Curve Fitting Toolbox.)

- Not-a-knot in B-spline form: `tau = augknt(x([1 3:end-2 end]), 4)`.
- `bsplvb` and `bsinterp` are what these do inside.

Summary: Why B-Splines

	truncated powers	B-splines
nonzero terms at a point	$O(n)$	$\leqslant k$
evaluation at a point	$O(n)$	$\frac{7}{2}k(k-1)$ flops
interpolation matrix	dense below diagonal	banded, width k
κ_∞, $k=4$, $n=80$	$9.2 \cdot 10^7$	4.8
meaning of coefficients	scaled jumps of $S^{(k-1)}$	de Boor points

Stable to compute, cheap to evaluate, easy to edit locally.

Looking Ahead

- **Next lecture:** Numerical Integration; Quadrature Rules